FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Calingasan, Dwayne Giusseppe P.
Section:	AN41
Professor:	Ms. Tan
Date:	10/03/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

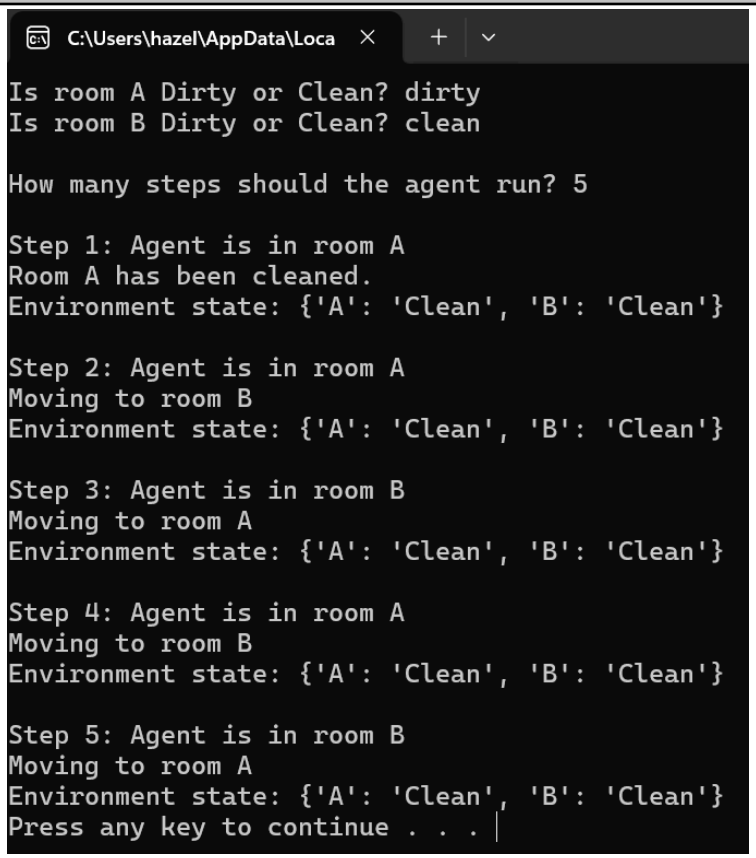
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

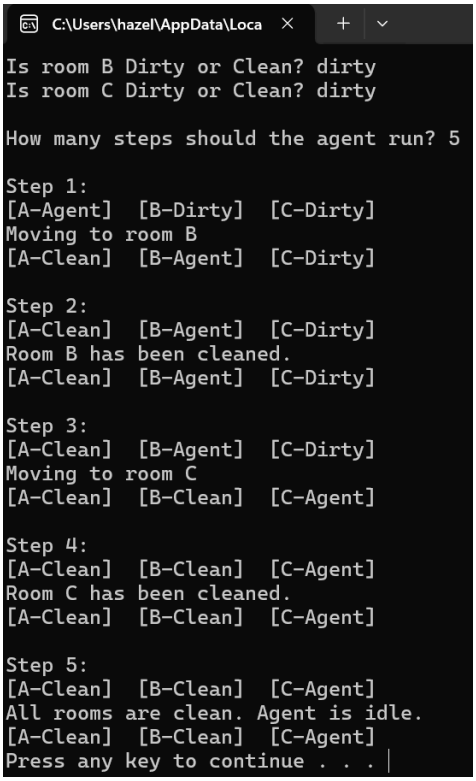
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
def is_dirty(environment, room):
    """Perceive whether the selected room needs cleaning."""
    return environment[room] == "Dirty"
```

```
def clean_room(environment, room):
    """Update the environment after a cleaning action."""
    environment[room] = "Clean"


def read_room_state(room):
    """Accept only Dirty or Clean, ignoring case and extra spaces."""
    while True:
        state = input(f"Is room {room} Dirty or Clean? ").strip().lower()
        if state in ("dirty", "clean"):
            return state.capitalize()
        print("Invalid state. Please enter Dirty or Clean.")


def read_steps():
    """Accept a non-negative whole number of simulation steps."""
    while True:
        try:
            steps = int(input("How many steps should the agent run? "))
            if steps >= 0:
                return steps
        except ValueError:
            pass
        print("Invalid steps. Please enter a whole number of 0 or more.")


class VacuumAgent:
    """A rule-based agent that starts in room A."""

    def __init__(self, environment):
        self.environment = environment
        self.location = "A"

    def move(self):
        """Switch from A to B or from B to A."""
```

```
        self.location = "B" if self.location == "A" else "A"

    def perceive_and_act(self):
        """Apply exactly one rule and perform one action per step."""
        if is_dirty(self.environment, self.location):
            # Rule 1: stay in the current room and clean it if dirty.
            clean_room(self.environment, self.location)
            return f"Cleaned room {self.location}"

        # Rule 2: move to the other room if the current room is clean.
        previous_room = self.location
        self.move()
        return f"Moved from room {previous_room} to room {self.location}"


def main():
    print("RULE-BASED VACUUM CLEANER AGENT")
    # Task 1: initialize the two-room dictionary from user input.
    environment = {room: read_room_state(room) for room in ("A", "B")}
    steps = read_steps()
    agent = VacuumAgent(environment)
    print(f"\nInitial environment: {environment}")
    print(f"Starting location: {agent.location}")

    # Task 3: execute the requested number of perceive-and-act cycles.
    for step in range(1, steps + 1):
        print(f"\nStep {step} | Location before action: {agent.location}")
        action = agent.perceive_and_act()
        print(f"Action: {action}")
        print(f"Current location: {agent.location}")
        print(f"Environment state: {environment}")

    # Continue moving even if both rooms are clean.
    print(f"\nSimulation complete: {steps} step(s).")
    print(f"Final location: {agent.location}")
    print(f"Final environment: {environment}")
```

```
if __name__ == "__main__":
    main()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two rules. Rule 1: If the current room is dirty, the agent cleans it and remains in that room. Rule 2: If the current room is clean, the agent moves to the other room. Only one action is performed per step.

2. The agent checks the state of its current room using is_dirty(). If the room is dirty, it cleans the room. If the room is clean, it moves to the other room. The decision is based only on the current room's state.

3. If both rooms are clean at the start, the agent will simply move between the rooms on each step: from A to B, then B to A, and so on, until the requested number of steps is reached. If the number of steps is zero, the agent performs no action and remains in room A.

4. I would add more rooms to the environment dictionary and store the room names in a list. The movement method could use an index to move through the rooms in sequence instead of switching only between A and B. The same cleaning rule would be applied to each room.

Screenshots:

